

73 - Open redirect vulnerability

Application Vulnerable to Open Redirect Attacks

- Your application redirects to a URL that's specified via the request such as the **querystring or form data**
- The redirection is performed without checking if the URL is a **local URL**

Към момента, и двете условия за уязвимост са изпълнени заради променливата return URL, без да имаме проверка дали URL е локален.

```
[HttpPost]
[AllowAnonymous]
0 references
public async Task<IActionResult> Login(LoginViewModel model, string returnUrl)
{
    if (ModelState.IsValid)
    {
        var result = await signInManager.PasswordSignInAsync(model.Email, model.Password,
                                                            model.RememberMe, false);

        if (result.Succeeded)
        {
            if (!string.IsNullOrEmpty(returnUrl))
            {
                LocalRedirect - in our application
                return Redirect(returnUrl);
                external, trusted URL
            }
            else
            {
                return RedirectToAction("index", "home");
            }
        }

        ModelState.AddModelError(string.Empty, "Invalid Login attempt");
    }
}
```

LocalRedirect – only to local URL If we provide an external URL – exception.

In Developer environment:

An unhandled exception occurred while processing the request.

InvalidOperationException: The supplied URL is not **local**. A URL with an absolute path is considered local if it does not have a host/authority part. URLs using virtual paths ('~/') are also local.

Microsoft.AspNetCore.Mvc.Infrastructure.LocalRedirectResultExecutor.ExecuteAsync(ActionContext context, LocalRedirectResult result)

Open Redirect Vulnerability Example

- The user is tricked into clicking a link in an email where the **returnUrl** is set to the attackers website

l -> I

<http://example.com/account/login?returnUrl=http://example.com/account/login>

- The user logs in successfully on the authentic site and he is then redirected to the attackers website
- The user logs in again on the attackers website, thinking that the first login attempt was unsuccessful
- The user is then redirected back to the authentic site
- During this entire process, the user does not know his credentials are stolen

```
if(!string.IsNullOrEmpty(returnUrl)) // && URL.IsLocalURL(returnUrl));  
{  
    return LocalRedirect(returnUrl);  
}
```

Prevent Open Redirect Attacks

```
public IActionResult Login(string returnUrl)  
{  
    return LocalRedirect(returnUrl);  
}
```

OR

```
public IActionResult Login(string returnUrl)  
{  
    if (Url.IsLocalUrl(returnUrl) && Url.IsLocalUrl(returnUrl))  
    {  
        return Redirect(returnUrl);  
    }  
    else  
    {  
        return RedirectToAction("index", "home");  
    }  
}
```

Link: <http://localhost:24077/account/login> -> index.